

Solving the one-dimensional bin packing problem with a weight annealing heuristic

Kok-Hua Loh^a, Bruce Golden^{a,*}, Edward Wasil^b

^a*R. H. Smith School of Business, University of Maryland, College Park, MD 20742, USA*

^b*Kogod School of Business, American University, Washington, DC 20016, USA*

Available online 28 December 2006

Abstract

In this paper, we develop a new procedure that uses the concept of weight annealing to solve the one-dimensional bin packing problem. Our procedure is straightforward and easy to follow. We apply it to 1587 instances taken from benchmark problem sets and compare our results to those found in the literature. We find that our procedure produces very high-quality solutions very quickly and generates several new optimal solutions.

© 2006 Elsevier Ltd. All rights reserved.

Keywords: Bin packing; Weight annealing; Heuristics; Combinatorial optimization

1. Introduction

In the one-dimensional bin packing problem (denoted by BPP), we have a set of $N = \{1, 2, \dots, n\}$ items that we need to pack into identical bins. Each item has a specified size and each bin has the same capacity. We want to find the minimum number of bins required to pack the items without violating the capacity constraint of each bin.

Over the last 15 years or so, a variety of exact methods and heuristic methods have been used to solve the BPP which is NP-hard [1]. Martello and Toth [2] solved the BPP with a branch-and-bound procedure (MTP). Scholl et al. [3] developed a hybrid method (BISON) that combines tabu search with a branch-and-bound procedure based on several bounds and a new branching scheme. Schwerin and Wäscher [4] devised a new lower bound for the BPP based on the cutting stock problem, integrated the new lower bound into MTP, and achieved high-quality results with their procedure (MTPCS).

Coffman et al. [5] provided a comprehensive review of approximation methods for bin packing including worst case analysis of the well-known first fit decreasing (FFD) and best fit decreasing (BFD) approximation algorithms. Recently, Fleszar and Hindi [6] developed a hybrid algorithm (denoted by PMBS' + VNS, where PMBS' denotes perturbation MBS') that combines a modified version of the minimal bin slack heuristic of Gupta and Ho [7] and variable neighborhood search, and uses the lower bounds of Fekete and Schepers [8]. Their hybrid algorithm performed very well in computational experiments—PMBS' + VNS found the optimal solutions to 1329 out of 1370 benchmark instances.

* Corresponding author.

E-mail address: bgolden@rhsmith.umd.edu (B. Golden).

Alvim et al. [9] solved the BPP with a sophisticated hybrid improvement heuristic (HI_BP) that uses tabu search to move items between bins. In computational experiments, HI_BP performed very well. It found optimal solutions to 1370 benchmark instances.

In this paper, we propose a simple and fast heuristic based on the concept of weight annealing (WA) to solve the BPP. WA is a new metaheuristic which has just recently been introduced in the physics literature. We believe that this paper is the first to present weight annealing to the operations research community. In Section 2, we describe WA and present similarities to other metaheuristics such as simulated annealing (SA). In Sections 3, 4, and 5, we describe the key features of WA for the BPP, our improvement heuristics, and our WA algorithm. In Section 6, we conduct extensive computational experiments with our WA algorithm and report results for eight sets of benchmark test problems taken from the literature. In Section 7, we summarize our contributions.

2. Weight annealing

Ninio and Schneider [10] proposed a WA method that allows a greedy heuristic to escape from a poor local optimum in the context of a combinatorial optimization problem. The key idea is changing the problem landscape and making use of the history of each optimization run. The authors changed the landscape by assigning weights to different parts of the solution space. Weights could be selected randomly or they could be selected to place strong emphasis on those parts of the solution space that are far from the optimal solution.

Ninio and Schneider provided the following outline of their WA algorithm.

Step 1: Start with an initial configuration from a greedy heuristic solution using the original problem landscape.

Step 2: Determine a new set of weights based on the previous optimization run and insight into the problem.

Step 3: Perform a new run of the greedy heuristic using the new weights.

Step 4: Return to Step 2 until a stopping criterion is met.

In their implementation of WA, Ninio and Schneider required nonnegative values for all of the weights so their algorithm could look for good solutions. They used a cooling schedule with temperature T to change the values of the weights. When the value of T was large, there were significant changes to the weights. As T decreased, all of the weights approached a value of one. Ninio and Schneider applied their WA algorithm to five benchmark traveling salesman problems (TSPs) with 127–1379 nodes and generated results that were competitive with SA.

WA shares features with metaheuristics such as SA (e.g., a cooling schedule) and deterministic annealing (DA). In particular, SA and DA start with an initial solution and perform a sequence of local search moves that include deteriorating moves. A deteriorating move may be allowed as long as the objective function value does not exceed a specified threshold. In contrast to SA and DA, WA not only considers the value of the objective function but also makes use of information on how well every part of the search space is being solved at every stage of an optimization run. When applied to a combinatorial optimization problem, WA seeks to expand and speed up the neighborhood search by creating distortions in different parts of the search space. The amount of the distortion is controlled by weight assignments based on insights gained from one iteration to the next, with a view towards focusing computational efforts on the poorly solved regions. We point out that Ninio and Schneider [10] provided a brief discussion of the similarities between WA and other algorithms including SA.

The notion of changing the landscape to produce high-quality solutions in WA has also been incorporated into several approaches including data smoothing (search space smoothing of Gu and Huang [11] and fine-tuned learning of Coy et al. [12]) and noising [13] for solving combinatorial optimization problems such as the TSP and the clique partitioning problem. For example, when using search space smoothing to solve the TSP, intercity distances are smoothed by a specified function and a local search heuristic is applied. The distances are smoothed again to a lesser extent and local search is performed again. This continues until the local search heuristic is applied to the problem with the original (unsmoothed) distances. By smoothing the distances, the problem landscape has been changed and hopefully a good solution in the smoothed space is a good solution to the original problem.

Ninio and Schneider [10] remarked that the noising method is the most closely related method to WA. Noising starts with a data set to which some noise has been added. A search method is applied until a local optimum is found. At each iteration of the method, less noise is added to the data set. At the final iteration, the original data set (with no noise added) is used. The best solution found during the process is the final solution.

Ninio and Schneider [10] pointed out that a “more elaborate version of WA which really considers how well the system is solved locally” could involve adaptive memory. In this case, WA would belong to a broad class of tabu search

methods. In addition, WA would be related to guided local search (GLS). GLS [14,15] uses memory to guide the search to those regions of the solution space that have promising solutions. This is accomplished by a term in the objective function that penalizes poor solutions.

3. Weight annealing for the bin packing problem

In this section, we describe the key features of WA for the BPP. These features include the objective function and weight assignments.

3.1. Objective function

In conducting our search, we use the objective function given by Fleszar and Hindi [6] that seeks to reduce the number of bins along with maximizing the sum of the squared bin loads:

$$\text{maximize } f = \sum_{i=1}^p (l_i)^2, \quad (1)$$

where p is the number of bins in the current solution and bin load l_i is the sum of sizes of items in bin i (that is, $l_i = \sum_{j=1}^{q_i} t_{ij}$, where t_{ij} is the size of item j in bin i and q_i is the number of items in bin i).

3.2. Weight assignments

A key feature of our procedure is the distortion of item sizes that allows for both uphill and downhill moves. The changes in the apparent sizes of the items are achieved by assigning different weights to the bins and their items according to how well the bins are packed.

For each bin i , we assign weight w_i^T according to

$$w_i^T = (1 + Kr_i)^T, \quad (2)$$

where C is the capacity of each bin, K is a constant, T is a temperature parameter (more about this in the next section), and $r_i = (C - l_i)/C$. The scaling parameter K controls the amount of size distortion for each item. The size distortion for an item is proportional to the residual capacity of its bin. At a local maximum, not-so-well packed bins will have large residual capacities. We try to escape from a poor local maximum with downhill moves. To enable downhill moves,

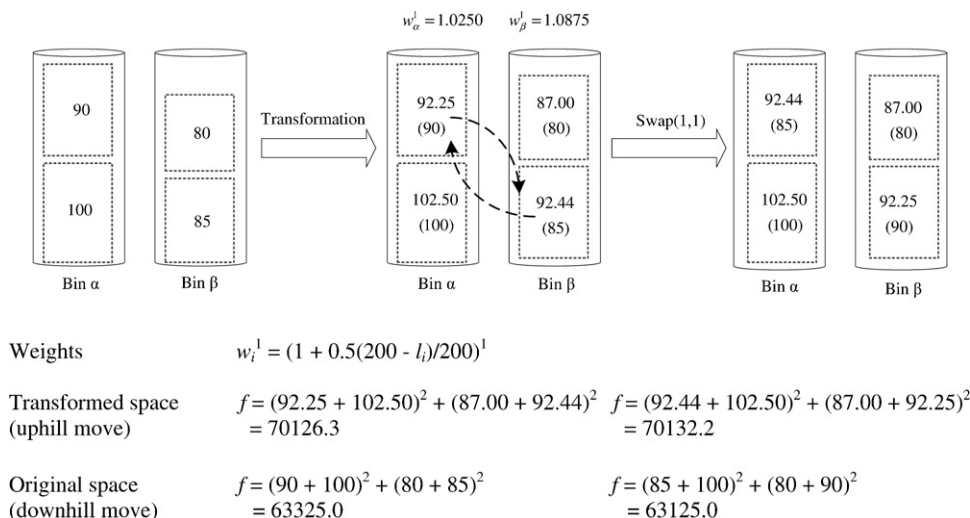


Fig. 1. An uphill move in the transformed space is a downhill move in the original space.

our weighting function increases the sizes of items in poorly packed bins. Since the objective function tries to maximize the number of fully filled bins, the size transformation increases the chances of a swap between one of the enlarged items in this bin and a smaller item from another bin. Thus, we have an uphill move in the transformed space, which may be a downhill move in the original space. We illustrate this case in Fig. 1 where $C = 200$. We make a swap as long as it is feasible in the original space.

4. Improvement heuristics: swap schemes

Neighbors of a current solution can be obtained by swapping (exchanging) items between all possible pairs of bins. We use four different swapping schemes: Swap (1,0), Swap (1,1), Swap (1,2), and Swap (2,2). We point out that the first two schemes were proposed by Fleszar and Hindi [6].

In Swap (1,0), we consider moving one item from bin α to bin β . The change in the objective function value ($\Delta f_{(1,0)}$) that results from moving one item i with size $t_{\alpha i}$ from bin α to bin β is given by

$$\Delta f_{(1,0)} = (l_{\alpha} - t_{\alpha i})^2 + (l_{\beta} + t_{\alpha i})^2 - l_{\alpha}^2 - l_{\beta}^2. \quad (3)$$

In Swap (1,1), we swap item i from bin α with item j from bin β . The change in the objective function value that results from swapping item i with size $t_{\alpha i}$ from bin α with item j with size $t_{\beta j}$ from bin β is given by

$$\Delta f_{(1,1)} = (l_{\alpha} - t_{\alpha i} + t_{\beta j})^2 + (l_{\beta} - t_{\beta j} + t_{\alpha i})^2 - l_{\alpha}^2 - l_{\beta}^2. \quad (4)$$

In Swap (1,2), we swap item i from bin α with items j and k from bin β . The change in the objective function value that results from swapping item i with size $t_{\alpha i}$ from bin α with item j with size $t_{\beta j}$ and item k with size $t_{\beta k}$ from bin β is given by

$$\Delta f_{(1,2)} = (l_{\alpha} - t_{\alpha i} + t_{\beta j} + t_{\beta k})^2 + (l_{\beta} - t_{\beta j} - t_{\beta k} + t_{\alpha i})^2 - l_{\alpha}^2 - l_{\beta}^2. \quad (5)$$

In Swap (2,2), we swap item i and item j from bin α with item k and item l from bin β . The change in the objective function value that results from swapping item i with size $t_{\alpha i}$ and item j with size $t_{\alpha j}$ from bin α with item k with size $t_{\beta k}$ and item l with size $t_{\beta l}$ from bin β is given by

$$\Delta f_{(2,2)} = (l_{\alpha} - t_{\alpha i} - t_{\alpha j} + t_{\beta k} + t_{\beta l})^2 + (l_{\beta} - t_{\beta k} - t_{\beta l} + t_{\alpha i} + t_{\alpha j})^2 - l_{\alpha}^2 - l_{\beta}^2. \quad (6)$$

5. Weight annealing algorithm for the bin packing problem

In Table 1, we present our WA algorithm for the one-dimensional BPP. We denote our algorithm by WABP.

WABP starts with an initial solution generated by the FFD procedure. For each bin i in the FFD solution, we compute the bin load (l_i) and the residual capacity (r_i).

To improve a solution, we carry out swapping operations with WA. A temperature parameter (T) controls the amount by which a single weight can be varied. At the start, a high temperature ($T = 1$) allows for higher frequencies of downhill moves. As the temperature is gradually cooled (the temperature is reduced at the end of every iteration, that is, $T \times 0.95$), the amount of item distortion decreases and the problem space looks more like the original problem space.

We compute a weight for each bin (according to $w_i^T = (1 + Kr_i)^T$) and then apply the weight to each item in the bin. The swapping process begins by comparing the items in the first bin with the items in the second bin, and so on, sequentially down to the last bin in the initial solution and is repeated for every possible pair of bins.

For a current pair of bins (α, β), the swapping of items by Swap (1,0) is carried out as follows. The algorithm evaluates whether the first item (item i) in bin α can be moved to bin β without violating the capacity constraint of bin β in the original space. In other words, does bin β have enough original residual capacity to accommodate the original size of item i ? If the answer is yes (the move is feasible), the change in objective function value of the move in the transformed space is evaluated. If $\Delta f_{(1,0)} \geq 0$, item i is moved from bin α to bin β . After this move, if bin α is empty and the total number of utilized bins reaches the lower bound, the algorithm stops and outputs the final results. If bin α is still partially filled, or the lower bound has not been reached, the algorithm exits Swap (1,0) and proceeds to Swap (1,1). If the move of the first item is infeasible or $\Delta f_{(1,0)} < 0$, the second item in bin α is evaluated and so on, until a feasible move

Table 1
Weight annealing algorithm for the one-dimensional bin packing problem

Step 0.	Initialization
	Parameters are K (scaling parameter), $nloop$, T (temperature), and $Tred$.
	Set $K = 0.05$, $nloop = 50$, $T = 1$, and $Tred = 0.95$.
	Inputs are number of items (n), item sizes (t_j), bin capacity (C), and lower bound (LB).
Step 1.	Construct an initial solution using First Fit Decreasing procedure.
	Sort the items in non-increasing order according to item sizes.
	Do while (item list is not empty)
	Place the first item in the list into the lowest numbered feasible bin.
	Open a new bin if the item cannot fit into any existing bin.
	Remove the item from the item list.
	Compute residual capacity r_i .
Step 2.	Improve the current solution.
	For $i = 1$ to $nloop$
	Compute the weights $w_i^T = (1 - Kr_i)^T$.
	Do for all pairs of bins
	Perform Swap (1,0)
	Evaluate feasibility and $\Delta f_{(1,0)}$.
	If $\Delta f_{(1,0)} \geq 0$
	Move the item.
	Exit Swap(1,0) and,
	Exit i loop if LB is reached.
	Exit Swap (1,0) if no feasible move with $\Delta f_{(1,0)} \geq 0$ is found.
	Perform Swap (1,1)
	Evaluate feasibility and $\Delta f_{(1,1)}$.
	If $\Delta f_{(1,1)} \geq 0$
	Swap the items.
	Exit Swap (1,1) and,
	Exit i loop if LB is reached.
	Exit Swap (1,1) if no feasible move with $\Delta f_{(1,1)} \geq 0$ is found.
	Perform Swap (1,2)
	Evaluate feasibility and $\Delta f_{(1,2)}$.
	If $\Delta f_{(1,2)} \geq 0$
	Swap the items.
	Exit Swap (1,2) and,
	Exit i loop if LB is reached.
	Exit Swap (1,2) if no feasible move with $\Delta f_{(1,2)} \geq 0$ is found.
	Perform Swap (2,2)
	Evaluate feasibility and $\Delta f_{(2,2)}$.
	If $\Delta f_{(2,2)} \geq 0$
	Swap the items.
	Exit Swap (2,2) and,
	Exit i loop if LB is reached.
	Exit Swap (2,2) if no feasible move with $\Delta f_{(2,2)} \geq 0$ is found.
	$T := T \times Tred$
	End of i loop
Step 3.	Output the results.
	Outputs are the number of bins used, the final distribution of items, and r_i .

with $\Delta f_{(1,0)} \geq 0$ is found or all items in bin α have been considered and no feasible move with $\Delta f_{(1,0)} \geq 0$ has been found. The algorithm then performs Swap (1,1), followed by Swap (1,2) and Swap (2,2). In each of the swapping schemes, we always take the first feasible move with $\Delta f \geq 0$ that we find.

We point out that the improvement step in WABP (Step 2) is carried out 50 times ($nloop = 50$) starting with $T = 1$, followed by $T = 1 \times 0.95 = 0.95$, $T = 0.95 \times 0.95 = 0.9025$, etc. We use a simple lower bound $\lceil (1/C) \sum_{j=1}^n t_j \rceil$, where t_j is the size of item j .

6. Computational results

In Table 2, we describe six well-known sets of benchmark test problems that contain 1587 problems. We coded WABP in C and C++ and then solved these test problems on a 3 GHz Pentium 4 computer with 256 MB of RAM.

In Table 3, we show the results for HI_BP, PMBS' + VNS, and WABP to 1370 instances from the Uniform, Triplet, and Set benchmark problem sets. The results for HI_BP and PMBS' + VNS are taken from the literature. The optimal solution is known for each instance, so we tabulate the number of times each procedure obtains the optimal solution. For example, there are 20 instances in problem set U120 and all three procedures find the optimal solution to each instance (there is a value of 20 in the Number Optimal column of each procedure). We see that both HI_BP and WABP found optimal solutions to all 1370 instances. PMBS' + VNS found optimal solutions to 1329 instances.

In Table 4, we show the results for BISON, HI_BP, PMBS' + VNS, MTPCS, and WABP to 1210 instances from the Set benchmark problem set. The results for HI_BP, BISON, PMBS' + VNS, and MTPCS are taken from the literature. The papers on BISON and MTPCS report results for the instances in Set and do not report results for instances in Uniform and Triplet. On Set 1 and Set 2 with MTPCS, Schwerin and Wäscher [4] set a time limit of 1000 s. This limit was insufficient for obtaining a lower bound for each instance in Set 3. The authors ran their lower bounding procedure for 1500–3000 s depending on the specific instance, started MTPCS with the lower bound, and set a time limit of 1000 s. We see that both HI_BP and WABP found optimal solutions to all 1210 instances. BISON, PMBS' + VNS, and MTPCS fell short and did not find the optimal solutions to 37, 40, and 94 instances, respectively.

In Tables 3 and 4, five different processors generated the results, ranging from a 66 MHz 80486 DX2 (used by BISON) which runs at 2.4 Mflop/s [20] to a 3 GHz Pentium 4 (used by WABP) which runs at 1573 Mflop/s [20]. The speeds of the remaining three processors fall between these extremes, so that there are roughly two to three orders

Table 2
Descriptions of benchmark test problems

Name	Notation	Comments
Uniform	U120, U250, U500, U1000	The bin capacity is $C = 150$. Items with integer sizes are drawn from a uniform distribution between 20 and 150. U120 denotes $n = 120$ items. For each value of $n = 120, 250, 500$, and 1000, there are 20 instances. These problems were developed by Falkenauer [16] and have been solved optimally by Carvalho [17]. They are available online at http://people.brunel.ac.uk/~mastjjb/jeb/orlib/binpackinfo.html .
Triplet	T60, T120, T249, T501	The bin capacity is $C = 1000$. Items with integer sizes are drawn from a uniform distribution between 250 and 500. T60 denotes $n = 60$ items. For each value of $n = 60, 120, 249$, and 501, there are 20 instances. The optimal solutions are known and have exactly three items per bin (hence the name triplets). These problems were developed by Falkenauer [16]. They are available online at http://people.brunel.ac.uk/~mastjjb/jeb/orlib/binpackinfo.html .
Set	Set 1, Set 2, Set 3	Set 1 has 720 instances with items drawn from a uniform distribution on three intervals [1, 100], [20, 100], and [30, 100]. The bin capacity is $C = 100, 120$, and 150 and $n = 50, 100, 200$, and 500. Set 2 has 480 instances with $C = 1000$ and $n = 50, 100, 200$, and 500. Each bin has an average of 3–9 items. Set 3 has 10 instances with $C = 100,000$, $n = 200$, and items are drawn from a uniform distribution on [20,000, 35,000]. Set 3 is considered the most difficult of the three sets. These problems were developed by Scholl et al. [3] and they reported that 1184 of the problems have been solved to optimality. Alvim et al. [9] reported the optimal solutions for the remaining 26 problems. The three problem sets are available online at http://paginas.fe.up.pt/~esicup/index.php .
Was	Was 1, Was 2	Was 1 has 100 instances with $C = 1000$ and $n = 100$. The minimum size of an item is 150 and the maximum size is 200. Was 2 has 100 instances with $C = 1000$ and $n = 120$. The minimum size of an item is 150 and the maximum size is 200. These problems were developed by Schwerin and Wäscher [4,18]. All problems have been solved to optimality. Both problem sets are available online at http://www.apdio.pt/sicup/Sicuphomepage/research.htm .
Gau	Gau 1	These 17 problems are taken from Wäscher and Gau [19]. They are reported as difficult problems by the authors. Some of these problems have been solved to optimality. This problem set is available online at http://paginas.fe.up.pt/~esicup/index.php .

Table 3

Results for HI_BP, PMBS' + VNS, and WABP to 1370 instances from the Uniform, Triplet, and Set benchmark problem sets

Name	Instances	HI_BP		PMBS' + VNS		WABP	
		Number optimal	Average time (s)	Number optimal	Average time (s)	Number optimal	Average time (s)
U120	20	20	0.00	20	0.02	20	0.00
U250	20	20	0.12	19	0.03	20	0.03
U500	20	20	0.00	20	0.04	20	0.18
U1000	20	20	0.01	20	0.07	20	1.24
T60	20	20	0.37	20	0.01	20	0.00
T120	20	20	0.85	20	0.02	20	0.00
T249	20	20	0.22	20	0.02	20	0.01
T501	20	20	2.49	20	0.06	20	0.03
Set1	720	720	0.19	694	0.15	720	0.17
Set2	480	480	0.01	474	0.10	480	0.19
Set3	10	10	4.60	2	3.74	10	0.13
Total	1370	1370		1329		1370	
Average			0.20		0.15		0.18
HI_BP	1.7 GHz Pentium 4						
PMBS' + VNS	400 MHz Pentium						
WABP	3 GHz Pentium 4						

Table 4

Results for BISON, HI_BP, PMBS' + VNS, MTPCS, and WABP to 1210 instances from the Set benchmark problem set

Name	Instances	BISON		HI_BP		PMBS' + VNS		MTPCS		WABP	
		Number optimal	Average time (s)	Number optimal	Average time (s)	Number optimal	Average time (s)	Number optimal	Average time (s)	Number optimal	Average time (s)
Set 1	720	697	32.4	720	0.19	694	0.15	717	7.3	720	0.17
Set 2	480	473	16.3	480	0.01	474	0.10	394	221	480	0.19
Set 3	10	3	700.2	10	4.60	2	3.74	5	3164.8	10	0.13
Total	1210	1173		1210		1170		1116		1210	
Average			31.53		0.16		0.15		118.2		0.18
BISON	66 MHz 80486 DX2										
MTPCS	90 MHz Pentium. Set 1 and Set 2 have a time limit of 1000 s. In Set 3, the lower bounding procedure is run anywhere from 1500 s to 3000 s and MTPCS is then started with the lower bound and given a time limit of 1000 s.										

of magnitude difference in the speeds of all five processors. Converting the running times of the four slower processors to a 3 GHz Pentium 4 time would yield dramatically smaller times in Tables 3 and 4.

In Table 5, we show the results for HI_BP and WABP to 200 instances from the Was benchmark problem set. The results for HI_BP are taken from the literature. The computation times for HI_BP were not reported. We note that Alvim et al. [9] reported that HI_BP improved the solutions to three instances (BPP56, BPP71, and BPP 81) in Was 1 and Was 2. We see that HI_BP and WABP found optimal solutions to all 200 instances. WABP took 0.01 s on average to solve an instance.

When Alvim et al. [9] applied HI_BP to 17 instances from the Gau benchmark set of problems, they reported that HI_BP found eight new, improved solutions and missed the optimal solutions to five instances. HI_BP was very fast with computation times of 0.01 s or less reported for the eight new solutions (no other times were given).

When MTPCS was applied to 17 instances from the Gau set, it generated the same best-known solutions to eight problems, found the optimal solution to one problem, and failed to find the optimal solutions to eight problems. The results for MTPCS were given at <http://www.apdio.pt/sicup/Sicuphomepage/research.htm>. The computation times for MTPCS were not reported on the web page. This page is no longer active.

Table 5

Results for HI_BP and WABP to 200 instances from the Was benchmark problem set

Name	Instances	HI_BP		WABP	
		Number optimal	Average time (s)	Number optimal	Average time (s)
Was 1	100	100	NA	100	0.01
Was 2	100	100	NA	100	0.01
Total		200		200	
NA	Not available				

Table 6

Results for WABP to 17 instances from the Gau benchmark problem set

Instance	Best Known	WABP		Instance	Optimal	WABP	
		Solution	Time (s)			Solution	Time (s)
TEST0005	29	29	0.03	TEST0044	14	14	0.08
TEST0014	24	24	0.00	TEST0049	11	11	0.03
TEST0022	15	15	0.00	TEST0054	14	14	0.03
TEST0030	28	28	0.05	TEST0055	15	15	0.03
TEST0058	21	20	0.20	TEST0055	20	20	0.19
TEST0065	16	16	0.00	TEST0075	13	13	0.33
TEST0068	13	12	0.54	TEST0084	16	16	0.00
TEST0082	25	24	0.00	TEST0095	16	16	0.03
				TEST0097	12	12	0.03

Bold: New optimal solution.

We applied WABP to 17 instances from the Gau set and present our results in Table 6. There are eight instances that have best-known solutions (listed on the left side of Table 6) and WABP found new, optimal solutions to three of these instances (TEST0058, TEST0068, and TEST0082). WABP generated the same best-known solutions to the remaining five instances. There are nine instances that have known optimal solutions (listed on the right side of Table 6) and WABP found the same optimal solutions to all nine instances. WABP took 0.09 s on average to solve an instance.

Finally, to evaluate the effect of WA on solving problems, we coded a multi-start procedure (denoted by MSP) that was capable of generating different initial solutions through random perturbations of the FFD algorithm. At every insertion step of FFD, we randomly selected one item out of a batch of the top 10% largest items. We then improved the solutions using the four swap schemes described in Section 4 and did not use any weights. We applied MSP to 20 instances in U1000, 10 instances in Set 3 (considered a difficult problem set), and 17 instances from the Gau set. MSP found the optimal solutions to all 20 instances in U1000 and all 10 instances in Set 3. WABP also found the optimal solutions to all 30 problems. On average, WABP was two to three times faster than MSP in solving these 30 problems. MSP found 10 optimal solutions and five best-known solutions to 17 instances from the Gau set. MSP did not find the optimal solutions to two instances. WABP found all 12 optimal solutions and five best-known solutions. We conclude that MSP is a very good procedure that benefits greatly from using our swap schemes. It is somewhat slower than WABP. Clearly, the use of weights in the improvement step of WABP (coupled with the swap schemes) is helpful. WABP is quickly guided towards optimal solutions and finds more optimal solutions than MSP.

7. Conclusions

We developed a powerful new procedure (WABP) that implements the concept of WA to solve the one-dimensional BPP problem. WABP is easy to understand and easy to follow, and it generated very high-quality solutions very quickly.

When applied to 1587 benchmark instances, WABP found the existing best known or optimal solutions to 1584 instances and generated new optimal solutions to the remaining three instances. Over all benchmark instances, it averaged 0.16 s in computation time. Our computational experiments showed that WABP performed slightly better

than a sophisticated procedure based on bounding and tabu search (HI_BP). It performed much better than a variable neighborhood search algorithm (PMBS'+ VNS) and a branch-and-bound procedure with a cutting stock lower bound (MTPCS).

In future work, we hope to apply our bin packing WA procedure to variants of the two-dimensional BPP.

References

- [1] Garey M, Johnson D. *Computers and intractability: a guide to the theory of NP-completeness*. San Francisco, CA: W.H. Freeman and Company; 1979.
- [2] Martello S, Toth P. *Knapsack problems: algorithms and computer implementations*. Chichester, England: Wiley; 1990.
- [3] Scholl A, Klein R, Jürgens C. BISON: a fast hybrid procedure for exactly solving the one-dimensional bin packing problem. *Computers & Operations Research* 1997;24:627–45.
- [4] Schwerin P, Wäscher G. A new lower bound for the bin-packing problem and its integration into MTP and BISON. *Pesquisa Operacional* 1999;19:111–29.
- [5] Coffman E, Garey M, Johnson D. Approximation algorithms for bin packing: a survey in: *Approximation algorithms for NP-hard problems* [D. Hochbaum, editor], vol. 46–93. Boston, MA: PWS Publishing; 1997.
- [6] Fleszar K, Hindi K. New heuristics for one-dimensional bin-packing. *Computers & Operations Research* 2002;29:821–39.
- [7] Gupta J, Ho J. A new heuristic algorithm for the one-dimensional bin packing problem. *Production Planning & Control* 1999;10:598–603.
- [8] Fekete S, Schepers J. New classes of lower bounds for the bin packing problem. *Mathematical Programming* 2001;91:11–31.
- [9] Alvim A, Ribeiro C, Glover F, Aloise D. A hybrid improvement heuristic for the one-dimensional bin packing problem. *Journal of Heuristics* 2004;10:205–29.
- [10] Ninio M, Schneider J. Weight annealing. *Physica A* 2005;349:649–66.
- [11] Gu J, Huang X. Efficient local search with search space smoothing: a case study of the traveling salesman problem. *IEEE Transactions on Systems, Man, and Cybernetics* 1994;24:728–35.
- [12] Coy S, Golden B, Runger G, Wasil E. See the forest before the trees: fine-tuned learning and its application to the traveling salesman problem. *IEEE Transactions on Systems, Man, and Cybernetics* 1998;28:454–64.
- [13] Charon I, Hudry O. The noising method: a new method for combinatorial optimization. *Operations Research Letters* 1993;14:133–7.
- [14] Voudouris C, Tsang E. Guided local search and its application to the traveling salesman problem. *European Journal of Operational Research* 1999;113:469–99.
- [15] Faroe O, Pisinger D, Zachariasen M. Guided local search for the three-dimensional bin-packing problem. *INFORMS Journal on Computing* 2003;15:267–83.
- [16] Falkenauer E. A hybrid grouping genetic algorithm for bin packing. *Journal of Heuristics* 1996;2:5–30.
- [17] Carvalho J. Exact solution of bin-packing problems using column generation and branch-and-bound. *Annals of Operations Research* 1999;86: 629–59.
- [18] Schwerin P, Wäscher G. The bin-packing problem: a problem generator and some numerical experiments with FFD packing and MTP. *International Transactions in Operational Research* 1997;4:377–89.
- [19] Wäscher G, Gau T. Heuristics for the integer one-dimensional cutting stock problem: a computational study. *OR Spektrum* 1996;18:131–44.
- [20] Dongarra J. Performance of various computers using standard linear equations software, Report CS-89-85, Computer Science Department, University of Tennessee, Knoxville, TN; 2006.